TRƯỜNG ĐẠI HỌC CÔNG NGHỆ – ĐHQGHN

HỌC PHẦN: LẬP TRÌNH HỆ THỐNG

LAB 6

TIẾN TRÌNH VÀ TÍN HIỆU TRONG LINUX

TÀI LIỆU HƯỚNG DẪN THỰC HÀNH

1 MỤC TIÊU HỌC TẬP

Sau khi hoàn thành nội dung này, sinh viên có thể:

- Phân biệt chương trình và tiến trình, giải thích PID, PPID, ngữ cảnh tiến trình và cây tiến trình.
- Sử dụng họ hàm `exec` để thay thế chương trình đang chạy bằng một chương trình khác.
- Sử dụng `wait()` và `waitpid()` để đồng bộ tiến trình cha – con và đọc trạng thái kết thúc của tiến trình con.
- Phân biệt tiến trình zombie và tiến trình mồ côi (orphan), đồng thời giải thích cách quan sát chúng trên Linux.
- Hiểu cơ chế tín hiệu và sử dụng `sigaction()` để xử lý một số tín hiệu cơ bản như `SIGINT`, `SIGTERM` và `SIGCHLD`.
- Xây dựng chương trình tạo tiến trình, chạy chương trình ngoài, chờ tiến trình con và kết thúc có kiểm soát.

2 CHƯƠNG TRÌNH VÀ TIẾN TRÌNH

2.1 Chương trình và tiến trình

Chương trình là tập hợp mã lệnh và dữ liệu được lưu trên thiết bị lưu trữ. Bản thân tệp thực thi chỉ là một đối tượng thụ động. Khi hệ điều hành nạp chương trình vào bộ nhớ và bắt đầu thực thi, một **tiến trình** được tạo ra.

Một chương trình có thể được chạy nhiều lần. Mỗi lần chạy tạo ra một tiến trình riêng với định danh, không gian địa chỉ và trạng thái thực thi riêng.

Khái niệm	Đặc điểm
Chương trình	Tệp chứa mã lệnh và dữ liệu, chưa thực thi, không có PID
Tiến trình	Một lần thực thi của chương trình, được nhân Linux quản lý và có PID riêng

Ví dụ

Nếu mở hai cửa sổ terminal và chạy cùng một chương trình ở cả hai cửa sổ, hệ điều hành quản lý hai tiến trình khác nhau dù chúng được tạo từ cùng một tệp thực thi.

2.2 Thành phần của tiến trình

Khi một chương trình đang chạy, tiến trình thường có các thành phần quan trọng sau:

- Mã lệnh đang thực thi
- Dữ liệu toàn cục và dữ liệu tĩnh
- Heap dùng cho bộ nhớ động
- Stack chứa lời gọi hàm và biến cục bộ
- Bảng bộ mô tả tệp đang mở
- Thư mục làm việc hiện tại, thông tin người dùng và các thiết lập tín hiệu

Các thành phần trên tạo nên trạng thái thực thi hiện tại của tiến trình. Khi chuyển CPU từ tiến trình này sang tiến trình khác, hệ điều hành phải lưu đủ thông tin để có thể tiếp tục tiến trình cũ ở đúng vị trí sau đó.

2.3 Ngữ cảnh tiến trình

Ngữ cảnh tiến trình có thể được hiểu là tập hợp thông tin cần thiết để hệ điều hành tạm dừng rồi tiếp tục tiến trình một cách chính xác.

Nhóm thông tin	Ví dụ
Trạng thái CPU	Program Counter, Stack Pointer, thanh ghi đa dụng, cờ trạng thái
Thông tin do hệ điều hành quản lý	PID, PPID, bản đồ bộ nhớ, bảng bộ mô tả tệp, thư mục làm việc, thiết lập tín hiệu

Việc lưu ngữ cảnh của một tiến trình và nạp ngữ cảnh của tiến trình khác được gọi là *chuyển ngữ cảnh* (context switch).

3 ĐỊNH DANH VÀ CÂY TIẾN TRÌNH

3.1 PID và PPID

Mỗi tiến trình đang tồn tại có một mã định danh gọi là PID. PID của tiến trình cha được biểu diễn qua PPID của tiến trình con.

Hai hàm thường dùng:

```
#include <unistd.h>

pid_t getpid(void);
pid_t getppid(void);
```

Hàm	Giá trị trả về
`getpid()`	PID của tiến trình đang gọi hàm
`getppid()`	PID của tiến trình cha hiện tại

Ví dụ:

```
#include <stdio.h>
#include <unistd.h>

int main(void) {
    printf("PID = %ld\\n", (long)getpid());
    printf("PPID = %ld\\n", (long)getppid());
    return 0;
}
```

3.2 Lệnh ps

`ps` hiển thị thông tin về các tiến trình. Trong Lab 6, cú pháp thường dùng là:

```
ps -o <danh_sach_cot>
ps -o <danh_sach_cot> -p <PID>
ps -ef
```

Ví dụ:

```
ps -o pid,ppid,stat,cmd
ps -o pid,ppid,stat,cmd -p 2412
ps -ef
```

Thành phần	Ý nghĩa
`-o`	Chọn các cột cần hiển thị
`pid`	PID của tiến trình
`ppid`	PID của tiến trình cha
`stat`	Trạng thái và một số cờ bổ sung của tiến trình
`cmd`	Câu lệnh hoặc tên chương trình đang chạy
`-p PID`	Chỉ hiển thị tiến trình có PID được chỉ định
`-e`	Chọn tất cả tiến trình
`-f`	Hiển thị định dạng đầy đủ hơn

3.3 Lệnh `pstree`

`pstree` biểu diễn quan hệ cha – con dưới dạng cây.

```
pstree
pstree -p
pstree -p <PID>
```

Trong đó:

- `pstree`: Hiển thị cây tiến trình.
- `-p`: Hiển thị thêm PID của từng tiến trình.
- `<PID>`: Chọn một tiến trình làm gốc của cây cần quan sát.

3.4 Cây tiến trình

Các tiến trình trong Linux hình thành quan hệ cha – con. Một tiến trình cha có thể tạo nhiều tiến trình con, và mỗi tiến trình con lại có thể tiếp tục tạo tiến trình mới.

Ghi nhớ

Kết thúc tiến trình cha không đồng nghĩa với việc tất cả tiến trình con tự động bị kết thúc. Nếu tiến trình cha kết thúc trước, các tiến trình con còn sống sẽ được hệ thống gán cho một tiến trình giám sát phù hợp.

4 TRẠNG THÁI TIẾN TRÌNH

Linux biểu diễn trạng thái tiến trình bằng các ký hiệu ngắn trong cột `STAT` của lệnh `ps`. Một số trạng thái thường gặp:

Ký hiệu	Tên	Ý nghĩa
`R`	Running/Runnable	Đang chạy trên CPU hoặc sẵn sàng được lập lịch
`S`	Sleeping	Đang chờ một sự kiện có thể bị đánh thức
`D`	Uninterruptible sleep	Thường đang chờ hoạt động trong nhân, chẳng hạn một số thao tác vào/ra
`T`	Stopped	Đang bị dừng, thường do tín hiệu hoặc công cụ gỡ lỗi
`Z`	Zombie	Đã kết thúc nhưng trạng thái kết thúc chưa được tiến trình cha thu thập

4.1 Lệnh `watch`

`watch` chạy lại một câu lệnh theo chu kỳ và cập nhật kết quả trên màn hình.

```
watch -n <so_giay> '<lenh>'
```

Ví dụ:

```
watch -n 0.5 'ps -o pid,ppid,stat,cmd -p <PID>'
```

Trong đó:

- `-n 0.5`: Chạy lại câu lệnh sau mỗi 0,5 giây.
- Chuỗi trong dấu nháy: Là câu lệnh được thực hiện lặp lại.
- `<PID>`: Thay bằng PID thực tế của tiến trình cần quan sát.

Trạng thái quan sát

Một tiến trình có thể chuyển trạng thái rất nhanh. Hai lần chạy `ps` liên tiếp có thể hiển thị hai trạng thái khác nhau cho cùng một PID.

5 TẠO TIẾN TRÌNH VỚI `fork()`

5.1 Cú pháp

`fork()` tạo một tiến trình con từ tiến trình đang gọi hàm.

```
#include <unistd.h>

pid_t fork(void);
```

`fork()` không nhận tham số. Kiểu trả về là `pid_t`, một kiểu số nguyên dùng để biểu diễn PID.

Sau lời gọi `fork()`, có ba trường hợp:

Giá trị trả về	Ý nghĩa
`-1`	Tạo tiến trình thất bại, không có tiến trình con mới
`0`	Mã đang chạy trong tiến trình con
Giá trị > 0	Mã đang chạy trong tiến trình cha, giá trị là PID của tiến trình con

5.2 Ví dụ cơ bản

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void) {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Child: PID=%ld PPID=%ld\n",
               (long)getpid(), (long)getppid());
    } else {
        printf("Parent: PID=%ld child=%ld\n",
               (long)getpid(), (long)pid);
    }

    return 0;
```

```
}
```

Cả tiến trình cha và tiến trình con đều tiếp tục thực thi từ lệnh ngay sau `fork()`. Điểm khác nhau quan trọng nhất để phân nhánh chương trình là giá trị trả về của `fork()`.

Biên dịch và chạy:

```
gcc -Wall -Wextra fork_demo.c -o fork_demo
./fork_demo
```

Nên chạy chương trình nhiều lần để quan sát rằng thứ tự in của cha và con có thể thay đổi.

Thứ tự thực thi

Sau `fork()`, bộ lập lịch của hệ điều hành quyết định tiến trình nào chạy trước. Nếu không có cơ chế đồng bộ, không nên giả định dòng của tiến trình cha hoặc tiến trình con sẽ xuất hiện trước.

5.3 Không gian địa chỉ

Về mặt logic, tiến trình con nhận một bản sao của không gian địa chỉ của tiến trình cha. Các biến có thể bắt đầu với cùng giá trị nhưng thuộc hai không gian địa chỉ khác nhau.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    int x = 5;
    pid_t pid = fork();

    if (pid == 0) {
        x = 9;
        printf("Child: x = %d\n", x);
        _exit(0);
    }

    waitpid(pid, NULL, 0);
    printf("Parent: x = %d\n", x);
    return 0;
}
```

Kết quả của tiến trình con là `9`, trong khi tiến trình cha vẫn giữ `5`.

5.4 Copy-on-Write

Linux thường không sao chép ngay toàn bộ các trang nhớ vật lý khi gọi `fork()`. Thay vào đó, tiến trình cha và con có thể tạm thời dùng chung các trang chỉ đọc. Khi một bên ghi vào một trang, hệ điều hành mới tạo một bản sao riêng cho bên đó. Cơ chế này gọi là *Copy-on-Write* (COW).

Nhờ COW, `fork()` có thể hoạt động hiệu quả ngay cả khi tiến trình có không gian địa chỉ lớn.

5.5 Bộ mô tả tệp

Bảng bộ mô tả tệp của tiến trình được sao chép cho tiến trình con. Tuy nhiên, các mục tương ứng trong hai bảng có thể cùng tham chiếu tới một *open file description* trong nhân Linux.

Điều này có nghĩa là tiến trình cha và tiến trình con có bộ mô tả riêng về mặt số nguyên, nhưng có thể dùng chung một số trạng thái ở mức nhân, đặc biệt là vị trí hiện tại trong tệp.

Vị trí tệp dùng chung

Nếu một tệp được mở trước `fork()`, sau đó cả tiến trình cha và con cùng dùng bộ mô tả được kế thừa, việc đọc hoặc ghi của một tiến trình có thể làm thay đổi vị trí tệp mà tiến trình còn lại quan sát được.

5.6 Vùng đệm `stdio`

Các vùng đệm của `stdio` thuộc không gian người dùng nên cũng được sao chép về mặt logic khi `fork()`. Nếu dữ liệu chưa được đẩy ra trước `fork()`, cả hai tiến trình có thể cùng mang bản sao của vùng đệm đó và cùng ghi nó sau này.

Khi cần tránh tình huống này, có thể gọi `fflush(NULL)` trước `fork()` nếu đang dùng `stdio`. Trong nhánh con dùng để gọi `exec()`, nếu `exec()` thất bại thì nên kết thúc bằng `_exit()` thay vì `exit()`.

6 KẾT THÚC TIẾN TRÌNH

Tiến trình có thể kết thúc bằng cách trả về từ hàm `main()`, gọi `exit()`, gọi `_exit()` hoặc bị kết thúc bởi tín hiệu.

6.1 Cú pháp

```c
#include <stdlib.h>

void exit(int status);
```

```c
#include <unistd.h>

void _exit(int status);
```

Trong đó:

- `status`: Mã trạng thái kết thúc được chuyển cho tiến trình cha.
- `exit()`: Kết thúc tiến trình sau khi thực hiện các thao tác dọn dẹp của thư viện C, bao gồm đẩy dữ liệu còn trong các stream chuẩn và gọi các hàm đã đăng ký bằng `atexit()`.
- `_exit()`: Kết thúc tiến trình ngay mà không thực hiện các thao tác dọn dẹp của thư viện C.

6.2 Phân biệt `exit()` và `_exit()`

Sau `fork()`, tiến trình con kế thừa trạng thái vùng đệm `stdio` của tiến trình cha. Vì vậy, trong nhánh con dùng để gọi `exec()`, nếu `exec()` thất bại thì thường sử dụng `_exit()` để tránh đẩy lại dữ liệu đã được kế thừa trong vùng đệm.

Ghi nhớ

`exit()` phù hợp khi chương trình muốn kết thúc theo cơ chế thông thường của thư viện C. `_exit()` thường được dùng trong tiến trình con sau `fork()` khi không muốn thực hiện lại các thao tác dọn dẹp ở mức thư viện.

7 HỌ HÀM `exec`

7.1 Nguyên lý hoạt động

Họ hàm `exec` không tạo tiến trình mới. Thay vào đó, nó nạp một chương trình mới vào **tiến trình hiện tại** và thay thế mã lệnh, dữ liệu, heap và stack của chương trình cũ.

Nếu `exec` thành công, hàm **không trả về**. Mã nguồn đặt sau lời gọi `exec` chỉ được thực thi khi lời gọi thất bại.

PID sau exec

Sau khi `exec` thành công, PID của tiến trình không đổi. Đây vẫn là cùng một tiến trình ở mức định danh, nhưng đang thực thi một chương trình mới.

7.2 Cú pháp

Các hàm thuộc họ `exec` được khai báo trong `<unistd.h>`. Dạng cơ sở là `execve()`:

```
#include <unistd.h>

int execve(const char *pathname,
           char *const argv[],
           char *const envp[]);
```

Các biến thể thường dùng:

```
int execl(const char *path,
          const char *arg0, ... /*, NULL */);

int execv(const char *path,
          char *const argv[]);

int execlp(const char *file,
           const char *arg0, ... /*, NULL */);

int execvp(const char *file,
           char *const argv[]);

int execle(const char *path,
           const char *arg0, ...
           /*, NULL, char *const envp[] */);
```

Ý nghĩa các ký tự trong tên hàm:

- Chữ `l`: Đối số được truyền dưới dạng danh sách.
- Chữ `v`: Đối số được truyền bằng mảng `argv[]`.
- Chữ `p`: Tệp thực thi được tìm trong các thư mục của biến môi trường `PATH`.
- Chữ `e`: Chương trình gọi hàm truyền tường minh mảng biến môi trường `envp[]`.

Các tham số chính:

- `path`: Đường dẫn đến tệp thực thi.
- `file`: Tên tệp thực thi. Với các hàm có chữ `p`, hệ thống tìm tệp theo `PATH`.
- `arg0`: Đối số đầu tiên, theo quy ước thường là tên chương trình.
- `argv[]`: Mảng các chuỗi đối số, phần tử cuối phải là `NULL`.
- `envp[]`: Mảng các chuỗi biến môi trường, phần tử cuối phải là `NULL`.

Nếu thành công, các hàm `exec` không trả về vì chương trình hiện tại đã được thay thế. Nếu thất bại, hàm trả về `-1` và đặt `errno`.

Hàm	Cách truyền đối số	Đường dẫn và môi trường
`execl()`	Danh sách	Dùng đường dẫn `path`
`execv()`	Mảng `argv[]`	Dùng đường dẫn `path`
`execlp()`	Danh sách	Tìm `file` theo `PATH`
`execvp()`	Mảng `argv[]`	Tìm `file` theo `PATH`
`execle()`	Danh sách	Dùng `path`, truyền thêm `envp[]`
`execve()`	Mảng `argv[]`	Dùng `pathname`, truyền `envp[]`

Trong các bài thực hành kiểu shell, `execvp()` thường thuận tiện vì câu lệnh đã được tách thành mảng `argv[]` và người dùng thường nhập tên lệnh như `ls` thay vì đường dẫn đầy đủ.

7.3 Hàm `execvp()`

Cú pháp:

```
#include <unistd.h>

int execvp(const char *file, char *const argv[]);
```

Ví dụ chạy lệnh `ls -l`:

```
char *argv[] = {"ls", "-l", NULL};
execvp(argv[0], argv);
perror("execvp");
```

Nếu `execvp()` thành công, dòng `perror()` không được thực thi. Nếu chương trình không tồn tại hoặc không thể thực thi, `execvp()` trả về `-1` và dòng xử lý lỗi được chạy.

7.4 Kết hợp `fork()` và `execvp()`

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        char *argv[] = {"ls", "-l", NULL};
        execvp(argv[0], argv);

        perror("execvp");
        _exit(127);
    }

    if (waitpid(pid, NULL, 0) == -1) {
        perror("waitpid");
        return 1;
    }

    return 0;
```

```
}
```

Quy trình điển hình là: Tiến trình cha gọi `fork()`, tiến trình con gọi `execvp()`, còn tiến trình cha dùng `waitpid()` để chờ tiến trình con.

7.5 Trạng thái tiến trình sau `exec`

Một số thuộc tính của tiến trình vẫn được giữ lại, chẳng hạn PID, PPID và thư mục làm việc hiện tại. Các bộ mô tả tệp đang mở cũng thường được giữ lại, trừ những bộ mô tả được đánh dấu đóng khi `exec` bằng cờ close-on-exec.

Các bộ xử lý tín hiệu đã được cài đặt bằng hàm sẽ được đưa về hành vi mặc định đối với nhiều tín hiệu khi `exec` thành công. Đây là một trong các lý do cần phân biệt rõ trạng thái của tiến trình và nội dung chương trình đang chạy trong tiến trình đó.

8 HỌ HÀM `wait`

8.1 Cú pháp

Tiến trình cha dùng `wait()` hoặc `waitpid()` để chờ và thu thập trạng thái của tiến trình con.

```
#include <sys/types.h>
#include <sys/wait.h>

pid_t wait(int *status);
pid_t waitpid(pid_t pid, int *status, int options);
```

Hàm `wait()`:

- Tham số `status`: Nhận trạng thái kết thúc của một tiến trình con. Có thể truyền `NULL` nếu không cần đọc trạng thái.
- Giá trị trả về: PID của tiến trình con đã được thu thập.
- Lỗi: Trả về `-1` và đặt `errno`.

Hàm `waitpid()`:

Tham số	Ý nghĩa trong Lab 6
`pid > 0`	Chờ đúng tiến trình con có PID bằng `pid`
`pid = -1`	Chờ một tiến trình con bất kỳ
`status`	Nhận trạng thái đã mã hóa, hoặc `NULL` nếu không cần
`options = 0`	Chờ theo cách thông thường, có thể làm tiến trình cha bị chặn
`options = WNOHANG`	Kiểm tra mà không chặn nếu chưa có tiến trình con sẵn sàng

Giá trị trả về của `waitpid()`:

- Giá trị > 0: PID của tiến trình con đã được thu thập.
- Giá trị `0`: Chỉ xuất hiện khi dùng `WNOHANG` và chưa có tiến trình con sẵn sàng.
- Giá trị `-1`: Có lỗi xảy ra.

8.2 Trạng thái kết thúc

Biến `status` chứa giá trị đã được mã hóa. Không nên tự tách các bit bằng phép toán thủ công. Thay vào đó, dùng các macro trong `<sys/wait.h>`.

Macro	Ý nghĩa
`WIFEXITED(status)`	Tiến trình con kết thúc bình thường
`WEXITSTATUS(status)`	Lấy mã kết thúc nếu `WIFEXITED` đúng
`WIFSIGNALED(status)`	Tiến trình con bị kết thúc bởi tín hiệu
`WTERMSIG(status)`	Lấy tín hiệu đã kết thúc tiến trình con

Ví dụ:

```
int status;
pid_t done = waitpid(pid, &status, 0);

if (done == -1) {
    perror("waitpid");
} else if (WIFEXITED(status)) {
    printf("Exit code = %d\n", WEXITSTATUS(status));
} else if (WIFSIGNALED(status)) {
    printf("Terminated by signal %d\n", WTERMSIG(status));
}
```

8.3 Đồng bộ tiến trình

Không có `waitpid()`, tiến trình cha và con có thể in kết quả theo thứ tự khác nhau giữa các lần chạy. Khi tiến trình cha gọi:

```
waitpid(pid, NULL, 0);
```

cha sẽ dừng tại đó cho đến khi tiến trình con tương ứng kết thúc. Nhờ vậy, các thao tác đặt sau `waitpid()` có thể được đảm bảo xảy ra sau tiến trình con.

8.4 Tùy chọn `WNOHANG`

```
pid_t r = waitpid(pid, &status, WNOHANG);
```

Khi dùng `WNOHANG`:

- Trả về PID của tiến trình con nếu đã có trạng thái để thu thập
- Trả về `0` nếu tiến trình con chưa thay đổi trạng thái phù hợp
- Trả về `-1` nếu xảy ra lỗi

8.5 Lỗi `EINTR`

Một số lời gọi chờ có thể bị tín hiệu làm gián đoạn và trả về lỗi `EINTR`. Mẫu xử lý chắc chắn hơn là thử lại khi lỗi này xuất hiện:

```
#include <errno.h>

pid_t r;
do {
    r = waitpid(pid, &status, 0);
} while (r == -1 && errno == EINTR);
```

9 ZOMBIE VÀ ORPHAN

9.1 Zombie

Khi tiến trình con kết thúc, nhân Linux giữ lại một lượng thông tin nhỏ về kết quả kết thúc để tiến trình cha có thể thu thập bằng `wait()` hoặc `waitpid()`.

Nếu tiến trình cha chưa thu thập thông tin đó, tiến trình con xuất hiện ở trạng thái `Z`. Đây là tiến trình zombie.

- Zombie không còn thực thi mã lệnh và không sử dụng CPU
- PID và một số thông tin trạng thái vẫn được giữ lại
- Nhiều zombie tồn tại lâu có thể làm lãng phí các mục quản lý tiến trình
- Cách xử lý cơ bản là tiến trình cha phải gọi `wait()` hoặc `waitpid()`

9.2 Orphan

Tiến trình mồ côi (orphan) là tiến trình con vẫn đang chạy nhưng tiến trình cha ban đầu đã kết thúc.

Linux sẽ gán tiến trình này cho một tiến trình giám sát phù hợp, chẳng hạn một child subreaper hoặc PID 1. Vì vậy, không nên viết quy tắc tuyệt đối rằng mọi orphan luôn có PPID bằng 1 trên mọi môi trường.

Phân biệt

Zombie là tiến trình đã kết thúc nhưng chưa được thu thập trạng thái. Orphan là tiến trình vẫn đang chạy nhưng tiến trình cha ban đầu đã kết thúc.

9.3 Quan sát zombie

Giả sử chương trình in PID của tiến trình con rồi để tiến trình cha ngủ một khoảng thời gian trước khi gọi `waitpid()`. Trong terminal thứ hai, dùng:

```
ps -o pid,ppid,stat,cmd -p <CHILD_PID>
```

Nếu cột `STAT` chứa `Z`, tiến trình con đang ở trạng thái zombie.

9.4 Quan sát orphan

Cho tiến trình cha kết thúc trước, còn tiến trình con gọi `sleep()` để tiếp tục tồn tại. Trong terminal thứ hai, dùng:

```
ps -o pid,ppid,stat,cmd -p <CHILD_PID>
```

Quan sát giá trị PPID trước và sau khi tiến trình cha kết thúc để thấy tiến trình con được gán cho tiến trình giám sát mới.

10 TÍN HIỆU

10.1 Khái niệm

Tín hiệu là một cơ chế thông báo bất đồng bộ tới tiến trình rằng một sự kiện đã xảy ra. Tín hiệu có thể đến từ terminal, nhân Linux hoặc một tiến trình khác.

Một số tín hiệu thường gặp:

Tín hiệu	Nguồn thường gặp	Ý nghĩa cơ bản
`SIGINT`	Nhấn Ctrl+C trong terminal	Yêu cầu ngắt tiến trình đang chạy ở foreground
`SIGTERM`	Lệnh `kill -TERM PID`	Yêu cầu tiến trình kết thúc, có thể được xử lý
`SIGKILL`	Lệnh `kill -KILL PID`	Buộc kết thúc, không thể bắt hoặc bỏ qua
`SIGCHLD`	Trạng thái của tiến trình con thay đổi	Báo cho tiến trình cha biết cần kiểm tra tiến trình con
`SIGSEGV`	Truy cập bộ nhớ không hợp lệ	Lỗi truy cập bộ nhớ nghiêm trọng

Nên sử dụng tên ký hiệu như `SIGTERM` thay vì ghi trực tiếp số tín hiệu vì số hiệu cụ thể có thể phụ thuộc hệ thống.

10.2 Lệnh `kill`

Tên lệnh `kill` có thể gây hiểu nhầm: Lệnh này không chỉ dùng để kết thúc tiến trình mà dùng để **gửi tín hiệu**.

Cú pháp thường dùng:

```
kill -<SIGNAL> <PID>
kill -s <SIGNAL> <PID>
kill -l
```

Ví dụ:

```
kill -TERM 2412
kill -INT 2412
kill -KILL 2412
kill -l
```

Trong đó:

- `<SIGNAL>`: Tên tín hiệu như `TERM`, `INT` hoặc `KILL`.
- `<PID>`: PID của tiến trình nhận tín hiệu.
- `-s`: Chỉ định tín hiệu bằng tên hoặc số.
- `-l`: Liệt kê các tín hiệu mà hệ thống hỗ trợ.

10.3 Hàm `kill()` và `raise()`

Trong chương trình C:

```c
#include <signal.h>
#include <sys/types.h>

int kill(pid_t pid, int sig);
int raise(int sig);
```

- `kill(pid, sig)`: Gửi tín hiệu `sig` tới tiến trình được xác định bởi `pid`.
- `raise(sig)`: Gửi tín hiệu `sig` cho chính tiến trình hiện tại.
- Giá trị trả về của `kill()`: Trả về `0` khi thành công, trả về `-1` khi có lỗi.
- Giá trị trả về của `raise()`: Trả về `0` khi thành công, khác `0` khi có lỗi.

10.4 Phản ứng với tín hiệu

Đối với nhiều tín hiệu, chương trình có thể chọn một trong ba cách:

- Sử dụng hành vi mặc định
- Bỏ qua tín hiệu nếu tín hiệu đó cho phép
- Cài đặt hàm xử lý tín hiệu

`SIGKILL` và `SIGSTOP` là hai ví dụ quan trọng không thể bị bắt hoặc bỏ qua.

10.5 Hàm `signal()`

Một giao diện đơn giản để đăng ký hàm xử lý tín hiệu là `signal()`:

```
#include <signal.h>

typedef void (*handler_t)(int);
handler_t signal(int sig, handler_t handler);
```

Trong đó:

- `sig`: Tín hiệu cần thiết lập cách xử lý.
- `handler`: Hàm xử lý tín hiệu, hoặc `SIG_DFL` để dùng hành vi mặc định, hoặc `SIG_IGN` để bỏ qua tín hiệu nếu được phép.

Trong Lab 6, `sigaction()` được ưu tiên vì cho phép mô tả đầy đủ hơn cách xử lý tín hiệu.

11 HÀM `sigaction()`

11.1 Cú pháp

`sigaction()` là giao diện nên dùng để thiết lập cách xử lý tín hiệu.

```
#include <signal.h>

int sigaction(int sig,
              const struct sigaction *act,
              struct sigaction *oldact);
```

Tham số	Ý nghĩa
`sig`	Tín hiệu cần thiết lập, ví dụ `SIGINT` hoặc `SIGTERM`
`act`	Con trỏ tới cấu trúc mô tả hành vi mới
`oldact`	Nhận hành vi cũ, hoặc truyền `NULL` nếu không cần

Hàm trả về `0` khi thành công và `-1` khi có lỗi.

11.2 Cấu trúc `sigaction`

Các trường thường dùng trong Lab 6:

```
struct sigaction sa = {0};
sa.sa_handler = handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;
```

Trong đó:

- `sa_handler`: Hàm xử lý được gọi khi tín hiệu đến, hoặc `SIG_DFL`/`SIG_IGN`.
- `sa_mask`: Tập tín hiệu tạm thời bị chặn trong lúc handler đang chạy.
- `sa_flags`: Các cờ bổ sung, ví dụ `SA_RESTART`.

- `sigemptyset()`: Khởi tạo một tập tín hiệu rỗng.

Ví dụ cài đặt handler cho `SIGTERM`:

```
struct sigaction sa = {0};
sa.sa_handler = on_term;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;

if (sigaction(SIGTERM, &sa, NULL) == -1) {
    perror("sigaction");
    return 1;
}
```

11.3 Hàm xử lý tín hiệu

Tín hiệu có thể đến vào bất kỳ thời điểm nào trong luồng thực thi thông thường. Vì vậy, hàm xử lý tín hiệu cần rất nhỏ và chỉ thực hiện công việc an toàn.

Một mẫu đơn giản là đặt một cờ kiểu `volatile sig_atomic_t`:

```
static volatile sig_atomic_t stop = 0;

static void on_term(int sig) {
    (void)sig;
    stop = 1;
}
```

Sau đó chương trình chính kiểm tra cờ và thực hiện các thao tác dọn dẹp trong luồng bình thường.

An toàn trong hàm xử lý tín hiệu

Các hàm như `printf()` hoặc `malloc()` không bảo đảm an toàn khi bị gọi bất đồng bộ trong signal handler. Trong các bài thực hành cơ bản, handler chỉ nên đặt cờ, còn việc in thông báo và giải phóng tài nguyên được thực hiện trong luồng chính.

11.4 Tín hiệu `SIGTERM`

```
#include <signal.h>
#include <stdio.h>
#include <unistd.h>

static volatile sig_atomic_t stop = 0;

static void on_term(int sig) {
    (void)sig;
    stop = 1;
}

int main(void) {
    struct sigaction sa = {0};
    sa.sa_handler = on_term;
    sigemptyset(&sa.sa_mask);

    if (sigaction(SIGTERM, &sa, NULL) == -1) {
        perror("sigaction");
        return 1;
    }

    printf("PID = %ld\n", (long)getpid());
```

```
    while (!stop)
        sleep(1);

    printf("Clean shutdown\n");
    return 0;
}
```

Từ terminal khác, có thể gửi tín hiệu bằng:

```
kill -TERM <PID>
```

11.5 Tùy chọn SA_RESTART

Một số lời gọi hệ thống có thể bị tín hiệu làm gián đoạn. Cờ SA_RESTART yêu cầu hệ thống tự động khởi động lại nhiều lời gọi bị gián đoạn khi phù hợp:

```
sa.sa_flags = SA_RESTART;
```

Không nên hiểu SA_RESTART là bảo đảm mọi lời gọi hệ thống đều tự động chạy lại. Chương trình vẫn cần kiểm tra giá trị trả về và xử lý lỗi phù hợp.

12 TÍN HIỆU SIGCHLD

Khi tiến trình con thay đổi trạng thái, tiến trình cha có thể nhận SIGCHLD. Một cách xử lý an toàn trong bài học cơ bản là để handler chỉ đặt cờ, sau đó phần mã thông thường gọi waitpid().

```
static volatile sig_atomic_t child_changed = 0;

static void on_chld(int sig) {
    (void)sig;
    child_changed = 1;
}
```

Khi cờ được đặt, có thể thu thập các tiến trình con đã kết thúc:

```
while (waitpid(-1, NULL, WNOHANG) > 0) {
    /* collect finished children */
}
child_changed = 0;
```

Đặc điểm của SIGCHLD

Nhiều sự kiện cùng loại có thể được gộp lại. Vì vậy, khi xử lý SIGCHLD, không nên giả định mỗi tín hiệu nhận được luôn tương ứng với đúng một tiến trình con vừa kết thúc. Việc lặp waitpid(-1, ..., WNOHANG) giúp thu thập tất cả tiến trình con đã sẵn sàng.

13 KẾT HỢP fork(), exec() VÀ waitpid()

Một mẫu thường gặp trong chương trình điều khiển tiến trình:

```
pid_t pid = fork();
if (pid < 0) {
    perror("fork");
    return 1;
}
```

```
if (pid == 0) {
    char *argv[] = {"date", NULL};
    execvp(argv[0], argv);
    perror("execvp");
    _exit(127);
}

int status;
if (waitpid(pid, &status, 0) == -1) {
    perror("waitpid");
    return 1;
}

if (WIFEXITED(status)) {
    printf("Child exit code = %d\n",
           WEXITSTATUS(status));
}
```

Mẫu này là nền tảng cho nhiều chương trình hệ thống: Tạo tiến trình con, thay thế chương trình trong tiến trình con và để tiến trình cha theo dõi kết quả.

14 LỖI THƯỜNG GẶP

1. Không kiểm tra trường hợp `fork()` trả về `-1`.
2. Giả định tiến trình cha luôn chạy trước hoặc tiến trình con luôn chạy trước.
3. Cho rằng biến của tiến trình cha và con vẫn là cùng một biến sau `fork()`.
4. Quên rằng bộ mô tả tệp kế thừa sau `fork()` có thể cùng tham chiếu tới một open file description.
5. Đặt logic cần thực hiện sau `exec()` mà không nhận ra rằng `exec()` không trả về khi thành công.
6. Dùng `exit()` trong nhánh con sau khi `exec()` thất bại mà không cân nhắc vùng đệm `stdio` được kế thừa.
7. Không gọi `wait()` hoặc `waitpid()` khiến tiến trình con trở thành zombie.
8. Đọc trực tiếp biến `status` của `waitpid()` thay vì dùng các macro `WIF...` và `W...STATUS`.
9. Gọi các hàm không an toàn như `printf()` hoặc `malloc()` trực tiếp trong signal handler.
10. Ghi cứng số hiệu tín hiệu thay vì dùng các tên như `SIGTERM` hoặc `SIGINT`.

15 TỔNG KẾT

Các nội dung chính cần ghi nhớ:

- Chương trình là tệp thụ động, tiến trình là một lần thực thi đang được hệ điều hành quản lý.
- `fork()` tạo tiến trình con, cha và con tiếp tục từ cùng vị trí nhưng nhận giá trị trả về khác nhau.
- Bộ mô tả tệp được kế thừa sau `fork()` và có thể dùng chung trạng thái của open file description, trong đó có file offset.
- `exec()` thay thế chương trình trong tiến trình hiện tại và không trả về khi thành công.
- `wait()` và `waitpid()` giúp đồng bộ và thu thập trạng thái kết thúc của tiến trình con.
- Zombie là tiến trình đã kết thúc nhưng chưa được cha thu thập trạng thái, còn orphan vẫn đang chạy nhưng cha ban đầu đã kết thúc.
- Tín hiệu là thông báo bất đồng bộ. Với các handler đơn giản, nên đặt cờ kiểu `sig_atomic_t` và xử lý công việc chính trong luồng bình thường.